



OpportunitySN
VEILLE EMPLOI TECH · SENEGAL

Veille automatisée du marché de l'emploi Tech au Senegal

Rapport de conception, de la premiere ligne au deploiement

Déployé sur : OpportunitySN — Veille marché emploi Tech

Papa Malick NDIAYE

Master Data Science et Genie Logiciel

Universite Alioune Diop de Bambey

GitHub : github.com/pa-malick

LinkedIn : linkedin.com/in/papa-malick-ndiaye-b58b22309

Sommaire

1. Introduction et objectifs
2. Comprendre la veille concurrentielle
3. Le choix du domaine : le marche de l'emploi Tech
4. L'architecture generale du projet
5. La conception etape par etape
6. Difficultes rencontrees et solutions
7. Ce que le projet demontre
8. Pistes d'amelioration
9. References

1. Introduction et objectifs

Ce rapport raconte la construction d'OpportunitySN, un projet personnel que j'ai monté pour renforcer mon profil de Data Scientist et Data Engineer. Avant tout, c'est un projet d'automatisation : l'objectif était de construire une pipeline de veille qui, une fois en ligne, collecte, nettoie, analyse et diffuse les données du marché de l'emploi toute seule, chaque jour, sans intervention.

Je ne voulais pas d'un ennemi notebook Jupyter isolé. Je voulais quelque chose qui ressemble à ce qu'on fait en entreprise : un système qui tourne tout seul, qui gère ses erreurs, qui se déploie, et qui produit un livrable lisible par un décideur non technique. Bref, passer de la donnée brute à la décision, automatiquement.

Le fil rouge : déployer une seule fois, puis laisser le système travailler seul, jour après jour.

Les objectifs métier

- Automatiser la veille de bout en bout : collecter, consolider, détecter les tendances.
- Produire un reporting intelligent : dashboard, email, alertes.
- Penser multi-sources et multi-marchés, comme un groupe qui opère dans plusieurs pays.
- Faciliter la décision : rendre l'information immédiatement actionnable.

Les objectifs d'apprentissage

- Maîtriser des outils modernes : scraping, ETL, Docker, Streamlit.
- Acquérir les réflexes du data engineering : validation, logging, gestion d'erreurs.
- Savoir documenter, déployer et communiquer à des non-techniciens.

Astuce : *Pour un projet de portfolio, ne cherchez pas la technologie la plus à la mode, mais celle que vous savez expliquer. En entretien, on vous demandera 'pourquoi ce choix', pas 'connais-tu tel outil'.*

2. Comprendre la veille concurrentielle

La veille concurrentielle, c'est l'art de surveiller son environnement pour prendre de meilleures décisions. Une entreprise veut savoir ce que font ses concurrents, comment bouge son marché, quelles compétences deviennent rares. Historiquement, ce travail se faisait à la main. Aujourd'hui, on l'automatise avec de la donnée.

Le point techno. Une pipeline de données suit presque toujours le même schéma : Extract (récupérer la donnée là où elle est), Transform (la nettoyer et la mettre en forme), Load (la stocker pour l'exploiter). C'est ce qu'on appelle un processus ETL. Dans son ouvrage de référence, Kimball décrit l'ETL comme le cœur caché de tout système décisionnel : peu visible, mais c'est là que se joue 70 pourcent du travail.

La litterature recente en ingenierie des donnees insiste sur un point : la valeur ne vient pas de la donnee brute, mais de la capacite a la transformer en information fiable et livree au bon moment. Reis et Housley, dans *Fundamentals of Data Engineering* (2022), parlent du cycle de vie de la donnee : generation, stockage, ingestion, transformation, puis service. Mon projet suit exactement ce cycle.

Astuce : *Quand vous presentez un projet data, racontez toujours le trajet de la donnee, pas la liste des outils. Un recruteur retient mieux 'de l'offre scrapee jusqu'a l'alerte Slack' qu'une liste de librairies.*

3. Le choix du domaine : le marche de l'emploi Tech

J'ai choisi le marche de l'emploi Tech au Senegal comme terrain d'application. Trois raisons a ce choix. D'abord, c'est un sujet que je connais bien puisque je cherche moi-meme a m'y positionner. Ensuite, les donnees sont publiques et abondantes : portails d'emploi, annonces, descriptions de poste. Enfin, c'est un domaine generique : la meme architecture marcherait pour de l'immobilier ou du e-commerce, il suffirait de changer les scrapers.

Concretement, le projet suit :

- Les offres qui apparaissent, par metier et par ville.
- Les competences les plus demandees, et lesquelles montent.
- Les salaires par categorie, avec detection des valeurs anormales.
- Les entreprises qui recrutent le plus.

4. L'architecture generale du projet

J'ai organise le code en cinq couches independantes. Chacune a une seule responsabilite et ne connait que la suivante. C'est un principe de conception simple mais puissant : on peut modifier une couche sans casser les autres.

Le flux de donnees ressemble a ceci :

- Scraping (Python) : on collecte les offres.
- ETL et validation (Pydantic) : on nettoie et on controle la qualite.
- Base de donnees (SQLite ou PostgreSQL) : on stocke avec historique.
- Analyse (anomalies et tendances) : on transforme en insights.
- Diffusion : dashboard Streamlit, email, et webhook Make vers Slack, Google Sheets et WhatsApp.

Le point techno. Ce decoupage en couches s'inspire du principe de separation des responsabilites. Chaque module fait une chose et la fait bien. C'est ce qui rend un projet maintenable : quand un bug apparait, on sait tout de suite dans quelle couche chercher.

5. La conception etape par etape

Etape 1 : mise en place et structure

J'ai commence par poser une structure de dossiers claire, un fichier de configuration central, et un systeme de logging partage. Ca peut paraitre secondaire, mais c'est ce qui evite le chaos plus tard. Toute la configuration (chemins, base de donnees, secrets) vit dans un seul endroit, et les secrets sont lus depuis un fichier .env jamais pousse sur GitHub.

Astuce : *Ne mettez jamais un mot de passe ou une cle directement dans le code. Un fichier .env plus un .gitignore, et le probleme est regle. C'est la premiere chose que regarde un relecteur attentif.*

Etape 2 : le scraping

Le scraping, c'est recuperer automatiquement des donnees depuis des pages web. J'ai construit une classe de base commune a tous mes scrapers, qui gere les requetes HTTP, les tentatives multiples en cas d'echec, le delai entre requetes, et la verification du robots.txt. Chaque source concrete n'a plus qu'a decire comment lire sa page.

Le point techno. Le fichier robots.txt d'un site indique ce que les robots ont le droit de visiter. Le respecter est une question d'ethique et de droit. De meme, on ajoute un delai entre les requetes (le throttling) pour ne pas surcharger le serveur. Mitchell, dans Web Scraping with Python (O'Reilly), insiste : un bon scraper est un scraper poli.

Point important : les sites changent souvent leur structure HTML et peuvent bloquer les robots. Pour que mon projet reste demonstrable en toutes circonstances, j'ai ajoute une source de demonstration qui genere un jeu de donnees realiste du marche senegalais. Ce n'est pas de la triche, c'est une source parmi d'autres qui garantit que la chaine complete tourne partout. Les vrais scrapers restent presents et fonctionnels.

Astuce : *Prevoyez toujours un plan B pour les donnees. Un projet qui plante parce qu'un site externe a change fait mauvais effet en demo. Une source stable de secours vous sauve la mise.*

Les sources reelles branchees

Au fil du projet, j'ai connecte cinq vraies sources d'offres, en plus de la source de demonstration. Deux passent par des API JSON publiques et documentees, RemoteOK et Arbeitnow, pour des postes Tech en teletravail. Trois sont senegalaises, Emploi Dakar, Expat-Dakar et Senjob, et fournissent des offres locales. Expat-Dakar affiche meme parfois de vrais salaires, precieux pour la detection d'anomalies.

J'ai vite appris une lecon de terrain : le scraping HTML direct est fragile. Plusieurs sites senegalais, dont EmploiSenegal, sont proteges par Cloudflare, qui bloque les robots simples avec une erreur 403. Plutot que de forcer le passage, j'ai privilegie des voies plus propres et plus stables : les API JSON quand elles existent, et le JSON-LD pour les autres.

Le point techno. Le JSON-LD est un bloc de données structurées, à la norme schema.org, que beaucoup de sites embarquent dans leurs pages. Pour une offre, on y trouve déjà le titre, l'entreprise, le lieu, la date, parfois le salaire, tout propre. Lire ce bloc est bien plus robuste que de parser le HTML à la main, car il ne casse pas quand le site change son design.

Comme ces sources généralistes renvoient tous les métiers, j'ai ajouté un filtre de pertinence qui ne garde que les postes Tech, pour rester fidèle à une veille emploi Tech.

Astuce : *Avant de vous acharner à contourner un site protégé, cherchez s'il expose une API ou du JSON-LD. On gagne du temps et on obtient des données plus fiables. Travailler avec le site plutôt que contre lui, c'est souvent la bonne stratégie.*

Etape 3 : l'ETL et la validation

Une fois les offres récupérées, elles sont sales : titres vides, salaires aberrants, doublons. L'étape ETL nettoie tout ça. Je devine la catégorie du métier à partir du titre, je retire les doublons, puis je fais passer chaque offre par un contrôle qualité avec Pydantic.

Le point techno. Pydantic valide les données à partir d'un schéma. On déclare la forme attendue d'une offre (types, valeurs par défaut, règles), et Pydantic rejette ou corrige ce qui ne colle pas. Concrètement, si un salaire minimum est supérieur au maximum, je remets les deux dans le bon ordre plutôt que de jeter l'offre. Après cette étape, toutes mes données ont la même forme, ce qui simplifie tout le reste.

Un cas concret rencontre en branchant les vraies sources : un nombre lu dans une description libre, un numéro par exemple, pouvait être pris pour un salaire de plusieurs milliards. J'ai donc ajouté un garde-fou qui écarte les valeurs aberrantes, au-delà d'un seuil plausible, et je n'extrais plus de salaire depuis du texte libre trop bruyant. Mieux vaut pas de salaire qu'un salaire faux.

Astuce : *Validez vos données le plus tôt possible, à l'entrée du système. C'est le principe du 'fail fast' : mieux vaut une erreur claire tout de suite qu'un bug mystérieux trois étapes plus loin.*

Etape 4 : le stockage

Les offres validées sont stockées en base avec SQLAlchemy. J'ai choisi SQLite par défaut, car il ne demande aucune installation : la base est un simple fichier. Pour passer à PostgreSQL en production, il suffit de changer une ligne dans le .env, car SQLAlchemy fait l'abstraction.

Chaque offre a un identifiant unique calculé à partir de son titre, de son entreprise et de sa source. Quand je re-scrape, je mets à jour l'offre existante au lieu de créer un doublon. Je garde aussi une date de collecte pour construire un historique et suivre l'évolution dans le temps.

Astuce : *SQLite est parfait pour démarrer et pour la plupart des projets de portfolio. Ne sortez PostgreSQL que quand vous avez vraiment plusieurs utilisateurs simultanés. Choisir simple est aussi une compétence.*

Etape 5 : l'analyse et la detection d'anomalies

C'est la partie la plus interessante scientifiquement. Je veux reperer les offres dont le salaire sort de l'ordinaire, soit une vraie opportunit , soit une erreur de saisie. J'utilise deux methodes statistiques complementaires.

Le point techno. Le Z-score mesure de combien d'ecarts-types une valeur s'eloigne de la moyenne. Au dela de 2.5, on considere la valeur comme rare. La methode IQR, elle, se base sur les quartiles : toute valeur en dehors de l'intervalle habituel est signalee. C'est la logique du boxplot, introduite par John Tukey dans Exploratory Data Analysis (1977). L'IQR est plus robuste quand la distribution n'est pas symetrique, ce qui est souvent le cas des salaires.

Je calcule tout par categorie de metier, car un salaire normal pour un Data Engineer n'a rien a voir avec celui d'un comptable debutant. Une offre est signalee si au moins une des deux methodes la detecte. Croiser les deux limite les faux negatifs.

La detection d'anomalies est un champ de recherche a part entiere. Chandola, Banerjee et Kumar, dans leur survey de reference (ACM Computing Surveys, 2009), rappellent qu'il n'existe pas de methode universelle : le bon choix depend de la nature des donnees. Pour des salaires, des methodes statistiques simples et interpretables sont souvent preferables a un modele complexe de type boite noire.

Astuce : *En entreprise, une methode simple qu'on peut expliquer bat souvent un modele complexe qu'on ne comprend pas. Si vous dites a un manager 'ce salaire est a trois ecarts-types de la moyenne du metier', il comprend. C'est ca, la valeur.*

Etape 6 : les alertes et notifications

Detecter, c'est bien, mais il faut prevenir les bonnes personnes. Je genere trois types d'alertes : les anomalies de salaire, les nouvelles offres dans les metiers que je suis en priorite, et les competences particulierement demandees. Chaque alerte a un niveau d'importance.

Les alertes partent par deux canaux. D'abord un rapport email en HTML, construit avec un template Jinja2, propre et lisible par un decideur. Ensuite, et c'est le point que je trouve le plus proche de la vraie vie, un webhook vers Make.

Le point techno. Make (ex-Integromat) et Zapier sont des outils no-code d'automatisation. Mon code Python envoie un simple message JSON a une URL de webhook Make. Ensuite, dans Make, je branche ce que je veux sans ecrire une ligne de code : un message Slack, une ligne dans un Google Sheet, un WhatsApp. C'est l'approche hybride code plus no-code qu'on retrouve partout en entreprise : le code fait le travail lourd et fiable, l'outil no-code gere la distribution.

Astuce : *Savoir combiner du code et des outils no-code est une competence tres appreciee. En entretien, pouvoir dire 'j'ai construit la pipeline en Python et connecte les alertes via Make, comme en entreprise' fait la difference.*

Ce canal n'est pas reste theorique : je l'ai reellement branche et teste de bout en bout. La pipeline envoie son message a Make, qui le relaie jusqu'a ma boite mail, et pourrait tout aussi bien alimenter Slack ou WhatsApp. J'ai vu les alertes arriver pour de vrai, avec les chiffres du jour, offres analysees, salaire median et competence la plus demandee. J'ai meme ajoute un formulaire d'abonnement sur le site : un visiteur laisse son email, un second scenario Make l'enregistre.

Etape 7 : le dashboard

Le dashboard est la vitrine pour un decideur. Je l'ai fait avec Streamlit, qui permet de creer une vraie application web en quelques dizaines de lignes, sans HTML ni JavaScript. On y retrouve les chiffres cles, les salaires par metier, les competences les plus demandees, les entreprises qui recrutent, l'evolution dans le temps, et la liste des anomalies. Le tout est filtrable par categorie et par ville, avec des graphiques interactifs Plotly.

Astuce : Mettez un cache sur le chargement des donnees d'un dashboard. Sans ca, chaque clic relit la base et l'appli devient lente. Un simple decorateur de cache change tout pour l'utilisateur.

En complement du dashboard Streamlit, j'ai construit une interface web dediee, plus proche d'un vrai outil utilisable au quotidien, avec Vue.js et une API FastAPI. La logique est simple : les filtres a gauche, les resultats a droite qui se recalculent en direct. Chaque offre s'ouvre dans un panneau de detail avec ses competences et un lien pour postuler, et les offres qui sortent du lot sont signalees d'un badge, opportunitie bien payee en vert, salaire suspect en rouge. L'API FastAPI sert les donnees au front, avec un instantane de secours pour que l'outil affiche toujours quelque chose, meme si le backend n'est pas lance.

Etape 8 : l'orchestration et l'automatisation

Une veille n'a d'interet que si elle tourne toute seule. J'ai prevu un planificateur simple avec la librairie schedule qui relance la pipeline chaque jour a 8h. Pour une version plus professionnelle, j'ai aussi ecrit un DAG Airflow, l'outil standard de l'industrie pour orchestrer des pipelines de donnees.

Le point techno. Airflow represente une pipeline comme un graphe de taches (un DAG, graphe oriente sans cycle). Chaque tache a ses dependances, ses tentatives, ses horaires. C'est plus lourd a mettre en place que schedule, mais indispensable des qu'une pipeline devient complexe. Proposer les deux montre que je sais choisir l'outil adapte a la taille du probleme.

Mais le vrai coup d'accelerateur, c'est l'automatisation complete du deploiement. J'ai mis en place une tache planifiee GitHub Actions qui, chaque matin a 08:00 et gratuitement, execute toute la boucle sans que personne ne touche a rien :

- elle relance le scraping sur les vraies sources ;
- elle rejoue l'ETL, la validation et l'analyse ;
- elle regenere l'instantane de donnees et le commit sur GitHub ;

- ce commit declenche le redéploiement automatique du site (Netlify) ;
- et le webhook Make envoie les alertes du jour.

Le resultat : on deploie le projet une seule fois, et il collecte, analyse, alerte et se met a jour tout seul, chaque jour. C'est un outil reellement vivant, pas une demo figee.

Astuce : *L'automatisation planifiee (cron GitHub Actions) est ce qui separe un script qu'on lance a la main d'un vrai produit qui tourne en continu. C'est le detail qui donne au projet une dimension 'ops' que les recruteurs remarquent.*

Etape 9 : Docker et deploiement

Pour que le projet tourne de la meme facon sur ma machine et en production, je l'ai conteneurise avec Docker. Un fichier docker-compose lance deux services : le dashboard et le planificateur. Le front, lui, se deploie gratuitement en statique sur Netlify ou Vercel, et lit l'instantane de donnees mis a jour automatiquement.

Le point techno. Docker emballe l'application et toutes ses dependances dans une image reproductible. Fini le fameux 'ca marche sur ma machine'. C'est devenu un standard incontournable du deploiement moderne.

Etape 10 : les tests et la qualite

J'ai ecrit des tests unitaires avec pytest pour les scrapers, le pipeline et l'analyse. Ils verifient par exemple que la validation rejette bien une offre sans titre, que le dedoublonnage fonctionne, ou que la detection d'anomalies repere une valeur volontairement aberrante. L'ensemble tourne automatiquement a chaque push grace a GitHub Actions.

Astuce : *Ne cherchez pas 100 pourcent de couverture de tests. Testez ce qui est important et ce qui casse facilement : la validation, les calculs, les cas limites. Un test qui verifie une evidence ne sert a rien.*

6. Difficultes rencontrees et solutions

Comme dans tout projet reel, j'ai rencontre quelques obstacles.

- Les sites d'emploi changent leur HTML et bloquent les robots. Solution : une source de demonstration stable en complement des vrais scrapers.
- Les donnees brutes sont sales et incoherentes. Solution : une validation stricte avec Pydantic, le plus tot possible dans la chaine.
- Distinguer un vrai salaire anormal d'une simple variation. Solution : croiser deux methodes statistiques et raisonner par categorie de metier.
- Diffuser vers plusieurs canaux sans reecrire du code a chaque fois. Solution : un webhook unique vers Make qui se charge de la distribution.
- Des sites comme EmploiSenegal et Emploi Dakar sont proteges par Cloudflare. Solution : passer par les API JSON et le JSON-LD des pages plutot que par le HTML brut, plus fragile et bloque.

- Un salaire lu dans du texte libre pouvait etre absurde, un numero pris pour un salaire de plusieurs milliards. Solution : un garde-fou qui ecarte les valeurs aberrantes des la validation.

7. Ce que le projet demontre

Au dela du resultat, ce projet montre un ensemble de competences concretes.

- Data engineering : ingestion multi-sources, ETL complet, validation, stockage.
- Analyse de donnees : statistiques, detection d'anomalies, analyse de tendances.
- Software engineering : architecture modulaire, tests, documentation, Git.
- Automation et DevOps : orchestration, cron GitHub Actions, Docker, integration continue, no-code.
- Communication : dashboard et rapport lisibles par un non-technicien.

En resume, je sais scraper, nettoyer, analyser, alerter et deployer, et surtout automatiser toute cette chaine de bout en bout. Je pense 'operations' et pas seulement 'science'. Et je sais expliquer chaque choix, ce qui est aussi important que le code lui-meme.

8. Pistes d'amelioration

- Ajouter d'autres sources senegalaises, notamment via Playwright pour les sites proteges par Cloudflare comme EmploiSenegal.
- Faire persister la base entre les executions planifiees, pour accumuler un vrai historique du marche.
- Entrainer un modele simple de prediction de salaire a partir des competences.
- Construire un historique versionne pour comparer semaine apres semaine.
- Ajouter des tests d'integration en plus des tests unitaires.
- Gerer plusieurs pays et devises pour une vraie dimension multi-marches.

9. References

- Kimball, R. et Ross, M. (2013). The Data Warehouse Toolkit, 3e edition. Wiley.
- Reis, J. et Housley, M. (2022). Fundamentals of Data Engineering. O'Reilly.
- Mitchell, R. (2018). Web Scraping with Python, 2e edition. O'Reilly.
- Tukey, J. W. (1977). Exploratory Data Analysis. Addison-Wesley.
- Chandola, V., Banerjee, A. et Kumar, V. (2009). Anomaly Detection: A Survey. ACM Computing Surveys, 41(3).
- Aggarwal, C. C. (2017). Outlier Analysis, 2e edition. Springer.
- Rousseeuw, P. J. et Hubert, M. (2011). Robust statistics for outlier detection. WIREs Data Mining and Knowledge Discovery.

OpportunitySN, projet realise par Papa Malick NDIAYE.